

React Context API

Teaching Session

Props · Prop Drilling · Context · useContext · Global Context

Simple examples, real understanding

What We'll Cover Today

01

Props — Quick Recap

What they are, how they flow

02

Prop Drilling

The problem we're here to solve

03

Context API

React's built-in solution

04

useContext Hook

Reading context in any component

05

Global Context Pattern

The clean, production-ready approach



01

Props — Quick Recap

Before we talk about the problem

What are Props?

Short for properties — how data flows from parent to child

Props = data passed from a **Parent** component to a **Child** component

```
// Parent — has the data
function App() {
  const username = 'Alice'
  return <Greeting name={username} />
}

// Child — receives via props
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>
}

// Output: Hello, Alice!
```

One-way

Parent → Child only

Read-only

Child cannot modify props

Any value

String, number, object, function



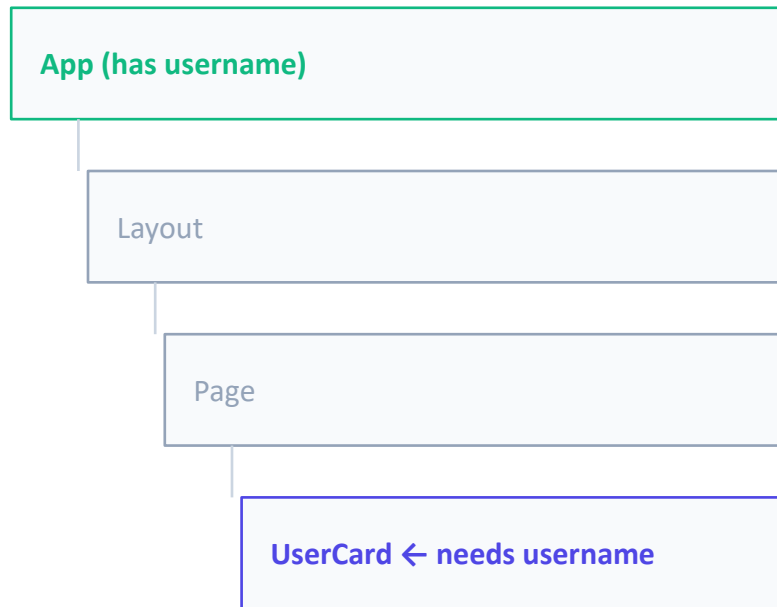
02

Prop Drilling

The problem that Context exists to solve

What is Prop Drilling?

Data passed through components that don't need it



Props passed down every level ↑

The Problem

- Layout and Page don't use username
- They receive it just to pass it along
- Add another prop? Change every level
- Rename it? Change everywhere
- 10 levels, 20 shared values = chaos

Prop Drilling in Code

Every middle component receives username just to pass it on

```
// username lives in App - UserCard needs it - 3 levels apart

function App() {
  const username = 'Alice'
  return <Layout username={username} /> // pass it down
}

function Layout({ username }) {          // doesn't use it
  return <Page username={username} />    // just passing it along
}

function Page({ username }) {            // doesn't use it
  return <UserCard username={username} /> // just passing it along
}

function UserCard({ username }) {        // finally uses it!
  return <p>Logged in as: {username}</p>
}
```



03

Context API

React's built-in solution to prop drilling

What is the Context API?

Any component can read the value directly — no passing through middle layers

Context = a broadcast signal. The Parent broadcasts a value. Any component tunes in directly.

`createContext()`

Creates the context channel

`createContext()`

`Context.Provider`

Broadcasts the value — wraps your components

`Context.Provider`

`useContext()`

Any component reads the value directly

`useContext()`

04

createContext · Provider · useContext

Three steps to use Context in your app

Step 1 — createContext()

Create the context channel — one line of code

```
// UserContext.js
import { createContext } from 'react'

const UserContext = createContext(null)

export default UserContext
```

What this does:

- Creates a Context object
- The default value (null) is used only if no Provider is above
- Export it so other files can use it
- By convention: name ends with Context

```
// The Context object has two things you need:
// 1. UserContext.Provider — to provide the value
// 2. useContext(UserContext) — to read the value
```

Step 2 — Context.Provider

Wrap your components. Pass the value. Middle components need nothing.

```
import UserContext from './UserContext'

function App() {
  const username = 'Alice'

  return (
    // Wrap with Provider — pass username as value
    <UserContext.Provider value={username}>
      <Layout />
      {/* No username prop needed here! */}
    </UserContext.Provider>
  )
}
```

 Before

```
<Layout username={username} />
```

 After

```
<Layout />    // clean — no props!
```

Step 3 — useContext()

Any component inside the Provider reads the value directly

```
import { useContext } from 'react'
import UserContext from './UserContext'

function UserCard() {
  // Read from context — no props!
  const username = useContext(UserContext)

  return <p>Logged in as: {username}</p>
}
```

How useContext works:

- 1 Takes the Context object as argument
- 2 Finds the nearest Provider above
- 3 Returns its current value
- 4 Re-renders when value changes

The Full Picture — No Props Through Middle Components

App (Provider) → Layout (nothing) → Page (nothing) → UserCard (useContext reads 'Alice')

Passing Multiple Values — Use an Object

Share data AND update functions through context

```
function App() {
  const [username, setUsername] = useState('Alice')

  // Pass object with value + setter
  return (
    <UserContext.Provider value={{ username, setUsername }}>
      <Layout />
    </UserContext.Provider>
  )
}
```

```
function UserCard() {
  const { username, setUsername } = useContext(UserContext)

  return (
    <div>
      <p>Hello, {username}</p>
      <button onClick={() => setUsername('Bob')}>
        Switch User
      </button>
    </div>
  )
}
```



Key:

When setUsername is called, App's state updates → Provider value updates → UserCard re-renders with new username automatically.



05

Global Context Pattern

One file per context — the clean way to organise

The Global Context Pattern

Keep everything for one context in one dedicated file

One file contains: createContext + Provider component (with state) + custom hook

createContext()

Creates the context channel

```
const ThemeContext =  
createContext(null)
```

Provider Component

Manages state, wraps children

```
export function ThemeProvider({  
  children })
```

Custom Hook

Hides useContext, clean API

```
export function useTheme()
```


ThemeContext.jsx — Complete File

Dark/light mode — the classic context example

```
import { createContext, useState,
        useContext } from 'react'

// Step 1: Create the context
const ThemeContext = createContext(null)

// Step 2: Provider manages state
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light')

  const toggleTheme = () => {
    setTheme(p => p==='light' ? 'dark':'light')
  }

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  )
}

// Step 3: Custom hook
export function useTheme() {
  return useContext(ThemeContext)
}
```

Why this structure?

Context kept private

Only the hook is exported — not the raw context

State lives here

ThemeProvider owns theme + toggleTheme

Custom hook = clean API

Components call useTheme(), not
useContext(ThemeContext)

One import

import { useTheme } from './ThemeContext'

Using Context in Components

Import once. Call the hook. Done.

main.jsx — wrap the app

```
import { ThemeProvider } from './ThemeContext'

<ThemeProvider>
  <App />
</ThemeProvider>
```

Navbar.jsx — uses toggle

```
import { useTheme } from './ThemeContext'

function Navbar() {
  const { theme, toggleTheme } = useTheme()

  return (
    <nav style={{
      background: theme==='dark'?'#111':'#eee'
    }}>
      <button onClick={toggleTheme}>
        Toggle Theme
      </button>
    </nav>
  )
}
```

Footer.jsx — reads theme only

```
import { useTheme } from './ThemeContext'

function Footer() {
  const { theme } = useTheme()

  return (
    <footer style={{
      color: theme==='dark'? 'white' : 'black'
    }}>
      © 2025 My App
    </footer>
  )
}
```



Navbar and Footer both read theme.
When toggleTheme is called, BOTH re-render.
No props. No drilling. Completely decoupled.

Multiple Contexts — Stack the Providers

One context per concern: User, Theme, Auth...

```
// main.jsx — stack providers
import { UserProvider } from './UserContext'
import { ThemeProvider } from './ThemeContext'
```

```
ReactDOM.createRoot(
  document.getElementById('root')
).render(
  <UserProvider>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </UserProvider>
)
```

```
// Any component can use both
import { useUser } from './UserContext'
import { useTheme } from './ThemeContext'
```

```
function Header() {
  const { user } = useUser()
  const { theme } = useTheme()

  return (
    <header style={{
      background:
        theme==='dark'?'#111':'#fff'
    }}>
      Welcome, {user.name}
    </header>
  )
}
```



One context per concern. UserContext handles user data. ThemeContext handles the theme. Keep them separate — easy to maintain.

Pro Tip — Add a Safety Guard

Catch the 'forgot to add Provider' error immediately

```
// Without guard - silent bug
export function useTheme() {
  return useContext(ThemeContext)
  // returns null if no Provider
  // theme.toggleTheme() → crashes
}

// With guard - clear error message
export function useTheme() {
  const context = useContext(ThemeContext)

  if (context === null) {
    throw new Error(
      'useTheme must be inside ThemeProvider'
    )
  }
}
```

Why bother?

- Without it: `null.theme` crashes with confusing error
- With it: 'useTheme must be inside ThemeProvider' — instantly clear
- Saves debugging time for you and your team
- Always add this in real projects

📌 Add this guard to every custom context hook you write. It takes 3 lines and saves hours of debugging.

Common Mistakes

Spot these before they trip you up

No Provider above component

→ Reads default (null) → silent bug



Always wrap in Provider in main.jsx

createContext inside component

→ Re-creates on every render



Call createContext at module level — outside any function

Not exporting custom hook

→ import {useTheme} fails



export function useTheme() { ... }

Single value, need multiple

→ Can't share setter



Pass object: value={{ theme, toggleTheme }}

Forgetting to import context

→ useContext(undefined)



Use the custom hook — it handles the import for you

Props vs Context vs Redux

Choosing the right tool for the job

Props

Use for: 1–2 levels deep

Clear, explicit data flow. Direct parent-child.

- ✓ Simple
- ✓ Readable
- ✓ Explicit

✗ Only one level at a time

Context

Use for: Many levels

Global data: theme, user, language, auth.

- ✓ No drilling
- ✓ Built into React
- ✓ Simple setup

✗ All consumers re-render on change

Redux / RTK

Use for: Large apps

Complex state, frequent updates, large teams.

- ✓ Powerful
- ✓ DevTools
- ✓ Predictable

✗ More setup needed

Quick Recap — Everything in One Slide

Props

One-way, parent → child. Read-only. Great for 1–2 levels.

Prop Drilling

Passing props through components that don't need them. Avoid it.

createContext()

Creates the context channel. Call at module level.

Context.Provider

Broadcasts the value. Wraps the part of your tree that needs it.

useContext()

Reads the value. Call inside any component inside the Provider.

Custom Hook

Wrap useContext in useTheme(), useUser(). Clean. Reusable.

The Context Pattern

`createContext()` → `Provider (value)` → `useContext()` → Custom Hook

createContext

Create the channel

Provider

Broadcast the value

useContext

Read it anywhere

Thank you